

A simple classifier example with the Iris dataset and LogisticRegression is shown below (<http://scikitlearnjl.readthedocs.io/en/latest/quickstart/>):

```
# The RDataset shall be added, if unavailable.
using RDatasets: dataset

iris = dataset("datasets", "iris")
# ScikitLearn.jl expects arrays, but DataFrames can also be
used - see
# the corresponding section of the manual
X = convert(Array, iris[[:SepalLength, :SepalWidth,
:Petallength, :PetalWidth]])
y = convert(Array, iris[:Species])

# Load the Logistic Regression model
using ScikitLearn
# This model requires scikit-learn. See
# http://scikitlearnjl.readthedocs.io/en/latest/
models/#installation
@sk_import linear_model: LogisticRegression
# The Hyperparameters such as regression strength, whether to
fit the intercept, penalty type.
model = LogisticRegression(fit_intercept=true)

# Train the model.
fit!(model, X, y)

# Accuracy is evaluated
accuracy = sum(predict(model, X) .== y) / length(y)
println("accuracy: $accuracy")
```

The cross-validation is shown in the code given below:

```
using ScikitLearn.CrossValidation: cross_val_score

cross_val_score(LogisticRegression(), X, y; cv=5) # 5-fold
> 5-element Array{Float64,1}:
> 1.0
> 0.966667
> 0.933333
> 0.9
> 1.0
```

ScikitLearn.jl may be used with other classifiers such as the *DecisionTree* classifier (*DecisionTree.jl* - <https://github.com/bensadeghi/DecisionTree.jl>). A random Forest based classifier is also shown below:

```
using RDatasets: dataset
using DecisionTree
iris = dataset("datasets", "iris")
features = convert(Array, iris[:, 1:4]);
labels = convert(Array, iris[:, 5]);
```

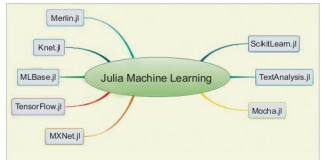


Figure 1: Machine learning libraries in Julia

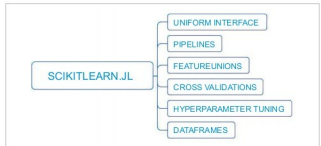


Figure 2: Features of *ScikitLearn.jl*

```
# train full-tree classifier
model = DecisionTreeClassifier(pruning_purity_threshold=0.9,
maxdepth=6)
fit!(model, features, labels)
# pretty print of the tree, to a depth of 5 nodes (optional)
print_tree(model.root, 5)
# apply learned model
predict(model, [5.9,3.0,5.1,1.9])
# get the probability of each label
predict_proba(model, [5.9,3.0,5.1,1.9])
println(get_classes(model)) # returns the ordering of the
columns in predict_proba's output
# run n-fold cross validation over 3 CV folds
# See ScikitLearn.jl for installation instructions
using ScikitLearn.CrossValidation: cross_val_score
accuracy = cross_val_score(model, features, labels, cv=3)
```

The following models are available in *DecisionTree.jl*:

- DecisionTreeClassifier
- DecisionTreeRegressor
- RandomForestClassifier
- RandomForestRegressor
- AdaBoostStumpClassifier

TensorFlow

TensorFlow is a recent buzzword in the machine learning world. This active open source machine learning framework from Google focuses on numerical computation with data flow graphs. The nodes represent mathematical operations, and the edges connecting the node indicate the data arrays termed *Tensors*. *TensorFlow.jl* is a Julia